



(12) 发明专利申请

(10) 申请公布号 CN 122387765 A

(43) 申请公布日 2026. 07. 14

(21) 申请号 202610856080.3

(22) 申请日 2026.06.15

(71) 申请人 上海燧原科技股份有限公司

地址 201306 上海市浦东新区中国(上海)
自由贸易试验区临港新片区业盛路
188号A-522室

(72) 发明人 卞劲松 周荣华 张文平 林川

(74) 专利代理机构 北京品源专利代理有限公司
11332

专利代理师 岳晓萍

(51) Int. Cl.

G06F 11/22 (2006.01)

G06F 11/263 (2006.01)

G06F 9/445 (2018.01)

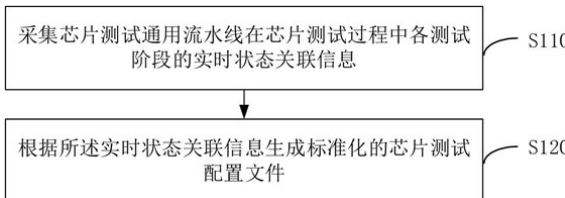
权利要求书2页 说明书18页 附图3页

(54) 发明名称

芯片测试配置文件生成方法、装置、设备、介质及程序

(57) 摘要

本发明实施例公开了一种芯片测试配置文件生成方法、装置、设备、介质及程序,其中,方法包括:采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;根据所述实时状态关联信息生成标准化的芯片测试配置文件。本发明实施例的技术方案能够提高芯片测试配置文件中配置信息的完整性、精确性和标准化程度,有利于实现芯片测试场景的标准化表达和精确复现。



1. 一种芯片测试配置文件生成方法,其特征在于,包括:
 - 采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;
 - 根据所述实时状态关联信息填充芯片测试配置文件的预设数据结构,得到初始芯片测试配置文件;
 - 根据所述初始芯片测试配置文件生成标准化的芯片测试配置文件;
 - 其中,所述根据所述实时状态关联信息填充芯片测试配置文件的预设数据结构,包括:
 - 从所述实时状态关联信息中提取元数据字段,并根据所述元数据字段填充所述芯片测试配置文件的元数据区结构;
 - 从所述实时状态关联信息中提取芯片硬件环境检测字段,并根据所述芯片硬件环境检测字段填充所述芯片测试配置文件的硬件环境描述区结构;
 - 从所述实时状态关联信息中提取芯片软件环境字段,并根据所述芯片软件环境字段填充所述芯片测试配置文件的软件环境描述区结构;
 - 从所述实时状态关联信息中提取环境配置字段,并根据所述环境配置字段填充所述芯片测试配置文件的环境配置区结构;
 - 从所述实时状态关联信息中提取测试执行字段,并根据所述测试执行字段填充所述芯片测试配置文件的测试执行描述区结构。
2. 根据权利要求1所述的方法,其特征在于,所述采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息,包括:
 - 在所述芯片测试通用流水线的环境准备阶段,采集芯片设备硬件信息;
 - 在所述芯片测试通用流水线的驱动安装阶段,采集芯片软件版本关联信息;
 - 在所述芯片测试通用流水线的软件配置阶段,采集环境变量快照信息;
 - 在所述芯片测试通用流水线的测试执行阶段,采集测试命令执行记录信息。
3. 根据权利要求1所述的方法,其特征在于,所述根据所述初始芯片测试配置文件生成标准化的芯片测试配置文件,包括:
 - 对所述初始芯片测试配置文件中的配置数据进行规范化验证;
 - 在确定所述初始芯片测试配置文件通过验证的情况下,将所述初始芯片测试配置文件进行序列化处理,得到所述标准化的芯片测试配置文件。
4. 根据权利要求3所述的方法,其特征在于,所述对所述初始芯片测试配置文件中的配置数据进行规范化验证,包括:
 - 对所述初始芯片测试配置文件中的必填字段的完整性进行验证;
 - 对所述初始芯片测试配置文件中的字段类型的正确性进行验证;
 - 对所述初始芯片测试配置文件中的芯片专用字段的有效性进行验证;
 - 对所述初始芯片测试配置文件中的测试场景必要字段的覆盖程度进行验证。
5. 根据权利要求3所述的方法,其特征在于,所述将所述初始芯片测试配置文件进行序列化处理,包括:
 - 将所述初始芯片测试配置文件按照设定标准格式进行格式化处理和缩进处理;
 - 对所述初始芯片测试配置文件添加配置文件生成时间戳和生成系统标识。
6. 根据权利要求1所述的方法,其特征在于,所述标准化的芯片测试配置文件覆盖的测试场景包括:核心转储分析场景、临时文件与工作目录场景、设备测试环境场景以及库应用

程序接口API测试场景中的至少一项。

7.一种芯片测试配置文件生成装置,其特征在于,包括:

实时状态关联信息采集模块,用于采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;

芯片测试配置文件生成模块,用于根据所述实时状态关联信息填充芯片测试配置文件的预设数据结构,得到初始芯片测试配置文件;根据所述初始芯片测试配置文件生成标准化的芯片测试配置文件;

所述芯片测试配置文件生成模块还用于:从所述实时状态关联信息中提取元数据字段,并根据所述元数据字段填充所述芯片测试配置文件的元数据区结构;从所述实时状态关联信息中提取芯片硬件环境检测字段,并根据所述芯片硬件环境检测字段填充所述芯片测试配置文件的硬件环境描述区结构;从所述实时状态关联信息中提取芯片软件环境字段,并根据所述芯片软件环境字段填充所述芯片测试配置文件的软件环境描述区结构;从所述实时状态关联信息中提取环境配置字段,并根据所述环境配置字段填充所述芯片测试配置文件的环境配置区结构;从所述实时状态关联信息中提取测试执行字段,并根据所述测试执行字段填充所述芯片测试配置文件的测试执行描述区结构。

8.一种电子设备,其特征在于,所述电子设备包括:

至少一个处理器;以及

与所述至少一个处理器通信连接的存储器;其中,

所述存储器存储有被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行权利要求1-6中任一所述的芯片测试配置文件生成方法。

9.一种计算机可读存储介质,其特征在于,所述计算机可读存储介质存储有计算机指令,所述计算机指令用于使处理器执行时实现权利要求1-6中任一所述的芯片测试配置文件生成方法。

10.一种计算机程序产品,其特征在于,包括计算机程序/指令,其中,所述计算机程序/指令被处理器执行时实现权利要求1-6中任一所述的芯片测试配置文件生成方法。

芯片测试配置文件生成方法、装置、设备、介质及程序

技术领域

[0001] 本发明实施例涉及芯片及测试技术领域,尤其涉及一种芯片测试配置文件生成方法、装置、电子设备、存储介质及程序。

背景技术

[0002] 芯片公司的软件栈由于其本身的复杂度较高,其测试流水线也相应地具有更高的复杂度。

[0003] 目前对芯片进行测试时,无论是普通的芯片还是AI(Artificial Intelligence, 人工智能)芯片,业界普遍采用CI(Continuous Integration,持续集成)/CD(Continuous Delivery,持续交付)流水线进行自动化测试。这种CI/CD流水线的自动化测试方法采用通用配置文件,如JSON(JavaScript Object Notation,JavaScript 对象表示法)或YAML(YAML Ain't Markup Language,一种轻量级且以数据为中心的序列化语言)等格式的通用配置文件,完成测试流程定义与参数配置。

[0004] 然而,现有芯片软件测试配置描述方法存在以下问题:(1)、缺乏芯片测试场景的领域特定结构抽象:现有芯片测试流程采用的通用配置文件仅提供数据序列化能力,不具备芯片测试领域的字段约束,无法表达加速卡运行模式(如物理/虚拟/切分)、固件版本约束以及功耗模式等芯片特有概念,缺乏对硬件-软件-依赖-执行多层依赖关系的结构化表达;(2)、测试环境描述的不完整性:传统测试脚本缺乏对硬件环境的精确描述能力,驱动安装参数及运行时配置等关键信息散落在多处,无法形成完整的环境快照用于问题复现;(3)、配置信息的非自描述性:现有芯片测试流程采用的通用配置文件缺乏数据模式版本控制机制,配置信息无法自解释其含义和用途,不同项目之间配置格式不统一,难以实现标准化。

发明内容

[0005] 本发明实施例提供一种芯片测试配置文件生成方法、装置、电子设备、存储介质及程序,能够提高芯片测试配置文件中配置信息的完整性、精确性和标准化程度,有利于实现芯片测试场景的标准化表达和精确复现。

[0006] 根据本发明的一方面,提供了一种芯片测试配置文件生成方法,包括:

[0007] 采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;

[0008] 根据所述实时状态关联信息生成标准化的芯片测试配置文件。

[0009] 根据本发明的另一方面,提供了一种芯片测试配置文件生成装置,包括:

[0010] 实时状态关联信息采集模块,用于采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;

[0011] 芯片测试配置文件生成模块,用于根据所述实时状态关联信息生成标准化的芯片测试配置文件。

[0012] 根据本发明的另一方面,提供了一种电子设备,所述电子设备包括:

[0013] 至少一个处理器;以及与所述至少一个处理器通信连接的存储器;其中,

[0014] 所述存储器存储有可被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行本发明任一实施例所述的芯片测试配置文件生成方法。

[0015] 根据本发明的另一方面,提供了一种计算机可读存储介质,所述计算机可读存储介质存储有计算机指令,所述计算机指令用于使处理器执行时实现本发明任一实施例所述的芯片测试配置文件生成方法。

[0016] 根据本发明的另一方面,还提供了一种计算机程序产品,包括计算机程序,所述计算机程序在被处理器执行时实现本发明任一实施例所述的芯片测试配置文件生成方法。

[0017] 本发明实施例通过采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息,从而根据采集的实时状态关联信息生成标准化的芯片测试配置文件,解决现有芯片测试场景中通用配置文件无法满足芯片专用测试需求的问题,能够提高芯片测试配置文件中配置信息的完整性、精确性和标准化程度,有利于实现芯片测试场景的标准化表达和精确复现。

[0018] 应当理解,本部分所描述的内容并非旨在标识本发明的实施例的关键或重要特征,也不用于限制本发明的范围。本发明的其它特征将通过以下的说明书而变得容易理解。

附图说明

[0019] 为了更清楚地说明本发明实施例中的技术方案,下面将对实施例描述中所需要使用的附图作简单地介绍,显而易见地,下面描述中的附图仅仅是本发明的一些实施例,对于本领域普通技术人员来讲,在不付出创造性劳动的前提下,还可以根据这些附图获得其他的附图。

[0020] 图1是本发明实施例一提供的一种芯片测试配置文件生成方法的流程图;

[0021] 图2是本发明实施例二提供的一种芯片测试配置文件生成方法的流程图;

[0022] 图3是本发明实施例提供的一种芯片测试配置文件生成系统的架构示意图;

[0023] 图4是本发明实施例提供的一种芯片测试配置文件的数据结构示意图;

[0024] 图5是本发明实施例三提供的一种芯片测试配置文件生成装置的示意图;

[0025] 图6为本发明实施例四提供的一种电子设备的结构示意图。

具体实施方式

[0026] 为了使本技术领域的人员更好地理解本发明方案,下面将结合本发明实施例中的附图,对本发明实施例中的技术方案进行清楚、完整地描述,显然,所描述的实施例仅仅是本发明一部分的实施例,而不是全部的实施例。基于本发明中的实施例,本领域普通技术人员在没有做出创造性劳动前提下所获得的所有其他实施例,都应当属于本发明保护的范围。

[0027] 需要说明的是,术语“包括”和“具有”及其任何变形,意图在于覆盖不排他的包含,例如,包含了一系列步骤或单元的过程、方法、系统、产品或设备不必限于清楚地列出的那些步骤或单元,而是可包括没有清楚地列出的或对于这些过程、方法、产品或设备固有的其它步骤或单元。

[0028] 实施例一

[0029] 图1是本发明实施例一提供的一种芯片测试配置文件生成方法的流程图,本实施例可适用于根据在芯片测试过程中采集的实时状态关联信息生成芯片专用测试配置文件的情况,该方法可以由芯片测试配置文件生成装置来执行,该装置可以由软件和/或硬件的方式来实现,并一般可集成在电子设备中,该电子设备可以是终端设备,也可以是服务器设备,只要能够执行芯片测试配置文件生成方法即可,本发明实施例并不对电子设备的具体设备类型进行限定。相应的,如图1所示,该方法包括如下操作:

[0030] S110、采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息。

[0031] 其中,芯片测试通用流水线可以是目前对芯片进行测试的通用流水线,例如可以是芯片的CI/CD流水线等,本发明实施例并不对芯片测试通用流水线的类型和测试流程进行限定。实时状态关联信息可以是在芯片测试通用流水线的各测试阶段实时采集的,多维度的状态信息。

[0032] S120、根据所述实时状态关联信息生成标准化的芯片测试配置文件。

[0033] 其中,芯片测试配置文件可以是专用于对芯片进行测试的配置文件。可选的,需要生成芯片测试配置文件的芯片可以是AI芯片,也还可以是普通的芯片,本发明实施例并不对芯片测试配置文件所适配的芯片类型进行限定。

[0034] 在本发明实施例中,可以通过配置实例生成工具,基于芯片测试通用流水线采集的信息自动化生成芯片专用的测试配置文件。其中,配置实例生成工具可以包括状态捕获模块和配置文件生成模块。状态捕获模块和配置文件生成模块之间保持通信连接。状态捕获模块可以负责采集芯片测试通用流水线在芯片测试过程中每个相关测试阶段的状态信息,并将采集的各测试阶段的状态信息进行整合,从而得到实时状态关联信息。示例性的,状态捕获模块可以在CI/CD流水线的环境准备、驱动安装、软件配置以及测试执行等各阶段实时采集状态信息。状态捕获模块可以将采集的实时状态关联信息存储在内部数据结构中,该内部数据结构例如可以为StateCollector(状态收集器)对象,并在流水线执行完成后统一传递给配置文件生成模块。进一步的,流水线执行完成后,状态捕获模块将实时状态关联信息发送至配置文件生成模块。配置文件生成模块可以对状态捕获模块发送的实时状态关联信息进行解析和整合,从而生成最终标准化的芯片测试配置文件。

[0035] 由于芯片测试通用流水线可能包括多个测试阶段,且每个测试阶段采集的状态信息可能包括多个类型。因此,状态捕获模块可以结合芯片测试需求,对实时状态关联信息进行分类。相应的,配置文件生成模块可以对每个类型的实时状态关联信息提取关键信息,并将提取的关键信息按照标准要求整理生成结构化的配置文件,从而得到标准化的芯片测试配置文件。相对于传统方法需要人工编写配置文件容易遗漏关键信息或产生不一致的技术问题,配置实例生成工具通过状态捕获模块和配置文件生成模块的相互配合,实现了从流水线执行到配置文件生成的自动转换,能够确保配置信息的完整性、准确性和一致性。

[0036] 本发明实施例通过采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息,从而根据采集的实时状态关联信息生成标准化的芯片测试配置文件,解决现有芯片测试场景中通用配置文件无法满足芯片专用测试需求的问题,能够提高芯片测试配置文件中配置信息的完整性、精确性和标准化程度,有利于实现芯片测试场景的标准

化表达和精确复现。

[0037] 实施例二

[0038] 图2是本发明实施例二提供的一种芯片测试配置文件生成方法的流程图,本实施例以上述实施例为基础进行具体化,在本实施例中,给出了采集实时状态关联信息以及生成标准化的芯片测试配置文件的多种具体可选的实现方式。相应的,如图2所示,本实施例的方法可以包括:

[0039] S210、在所述芯片测试通用流水线的环境准备阶段,采集芯片设备硬件信息。

[0040] 其中,芯片测试通用流水线的环境准备阶段是确保测试过程高精度及高可靠性的基础环节,主要包含测试环境、设备、样品和方案的全面就绪准备工作。

[0041] 可选的,状态捕获模块可以包括硬件信息采集器、软件版本记录器、环境变量快照器以及命令执行记录器等多种类型的数据采集器。其中,硬件信息采集器可以用于采集芯片的硬件信息,如GCU (General Computing Unit,通用计算单元) 设备ID、数量以及运行模式等。软件版本记录器可以用于记录驱动版本、运行时版本以及容器镜像版本等信息。环境变量快照器可以用于捕获宿主机和容器内的关键环境变量等信息。命令执行记录器可以用于记录各阶段执行的命令序列,如芯片检测命令:target_check_cmd和驱动安装命令:kmd_install_cmd等。

[0042] 具体的,硬件信息采集器可以在芯片测试通用流水线的环境准备阶段采集芯片设备硬件信息,如通过lspci (List PCI devices,列出PCI设备) 命令检测GCU设备ID和数量等。可选的,在环境准备阶段,状态捕获模块还可以通过环境变量快照器来捕获宿主机环境变量,并通过命令执行记录器来记录系统检查命令如system_check_cmd和芯片检测命令如target_check_cmd。

[0043] S220、在所述芯片测试通用流水线的驱动安装阶段,采集芯片软件版本关联信息。

[0044] 其中,芯片测试通用流水线的驱动安装阶段是确保自动化测试设备与主机系统之间稳定通信的关键环节。该阶段主要涉及调试器/编程器驱动、USB (Universal Serial Bus,通用串行总线) 转串口驱动以及芯片厂商专用驱动的正确部署。芯片软件版本关联信息可以为芯片的软件版本所相关联的信息,例如可以包括但不限于驱动版本号、驱动安装命令及驱动参数等。

[0045] 相应的,状态捕获模块可以在芯片测试通用流水线的驱动安装阶段,通过软件版本记录器和命令执行记录器采集存储芯片软件版本关联信息。例如,状态捕获模块可以通过软件版本记录器记录驱动版本信息,并通过命令执行记录器记录驱动安装命令如kmd_install_cmd及该驱动安装命令包括的驱动安装参数等。

[0046] S230、在所述芯片测试通用流水线的软件配置阶段,采集环境变量快照信息。

[0047] 其中,芯片测试通用流水线的软件配置阶段主要涉及测试程序、测试环境、自动化脚本及工具链的初始化与参数化设置,目的是确保测试流程能够高效且准确地执行芯片功能与性能验证。

[0048] 相应的,状态捕获模块可以在芯片测试通用流水线的软件配置阶段,通过软件版本记录器和命令执行记录器采集存储芯片的环境变量快照信息。可选的,环境变量快照信息可以包括但不限于容器镜像版本、容器启动参数、SDK (Software Development Kit,软件开发工具包) 版本以及环境变量设置命令等。例如,状态捕获模块可以通过软件版本记录器

记录芯片的容器镜像版本和SDK版本等信息,并通过命令执行记录器记录容器启动命令如docker_launch_cmd和容器内环境设置命令等,如通过SDK安装时,容器内环境设置命令可以为container_env_setup。

[0049] S240、在所述芯片测试通用流水线的测试执行阶段,采集测试命令执行记录信息。

[0050] 其中,芯片测试通用流水线的测试执行阶段主要包括晶圆测试、封装测试和系统级测试等,以验证芯片的功能、性能和可靠性。测试命令执行记录信息可以包括但不限于测试命令序列以及核心转储文件路径等与测试命令的执行情况相关的信息。

[0051] 相应的,状态捕获模块可以在芯片测试通用流水线的测试执行阶段,通过命令执行记录器记录存储芯片的各项测试命令信息。示例性的,状态捕获模块可以通过命令执行记录器记录存储测试命令前缀以及各种类型的测试命令构成的测试命令序列。如果芯片在芯片测试通用流水线的测试执行阶段发生崩溃,还可以记录用于崩溃分析的核心转储文件路径。核心转储文件路径是指测试流程在进程异常终止时,将测试状态保存到磁盘的指定位置。

[0052] 由此可见,状态捕获模块通过各类数据采集器在芯片测试通用流水线的各阶段互相协作,以采集芯片测试配置文件所需的全量状态信息。

[0053] S250、根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,得到初始芯片测试配置文件。

[0054] 其中,预设数据结构可以是考虑芯片特性,针对芯片测试配置文件所特定配置的专用数据结构。初始芯片测试配置文件可以是基于实时状态关联信息和芯片测试配置文件的预设数据结构所初步生成的芯片测试配置文件。

[0055] 在本发明实施例中,配置文件生成模块接收到状态捕获模块采集的全量状态信息,通过模板填充引擎、模式验证器和文件序列化器三个组件协作生成标准化的芯片测试配置文件。相应的,配置文件生成模块可以通过模板填充引擎对接收的实时状态关联信息填充到芯片测试配置文件的预设数据结构中,得到初始芯片测试配置文件。

[0056] 在本发明的一个可选实施例中,所述根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,可以包括:从所述实时状态关联信息中提取元数据字段,并根据所述元数据字段填充所述芯片测试配置文件的元数据区结构;从所述实时状态关联信息中提取芯片硬件环境检测字段,并根据所述芯片硬件环境检测字段填充所述芯片测试配置文件的硬件环境描述区结构;从所述实时状态关联信息中提取芯片软件环境字段,并根据所述芯片软件环境字段填充所述芯片测试配置文件的软件环境描述区结构;从所述实时状态关联信息中提取环境配置字段,并根据所述环境配置字段填充所述芯片测试配置文件的环境配置区结构;从所述实时状态关联信息中提取测试执行字段,并根据所述测试执行字段填充所述芯片测试配置文件的测试执行描述区结构。

[0057] 其中,元数据是描述数据的数据,它不包含数据本身的内容,而是提供关于数据的结构、含义、来源和用途等关键信息,帮助人们和系统理解、管理、发现和使用数据。元数据字段可以是能够描述元数据的相关字段。元数据区结构(可简称为元数据区)可以是预设数据结构中用于配置文件的版本控制和溯源管理的数据结构。芯片硬件环境检测数据主要指在芯片运行或测试过程中,对其所处物理环境及自身工作状态的实时监测参数,用于保障稳定性、可靠性与安全性。这些数据通常由片上传感器或外部监控系统采集,涵盖温度、电

压、功耗、湿度和振动等多个维度。芯片硬件环境检测字段可以是能够描述芯片硬件环境检测数据的相关字段。硬件环境描述区结构(可简称为硬件环境描述区)可以是预设数据结构中专门用于描述芯片硬件检测逻辑的数据结构。芯片软件环境数据是指在芯片运行、测试或开发过程中,与其软件系统密切相关的配置、状态和行为信息,用于确保软件与硬件协同工作正常,并支撑功能验证、性能调优与故障诊断。芯片软件环境字段可以是能够描述芯片软件环境数据的相关字段。软件环境描述区结构(可简称为软件环境描述区)可以为用于描述芯片驱动、运行时库和容器环境的数据结构。环境配置数据是确保芯片在开发、测试与运行阶段能够稳定工作的关键参数集合,涵盖硬件、软件及物理环境的设定信息,直接影响系统兼容性、性能表现与可靠性。环境配置字段可以为用于描述环境配置数据的相关字段。环境配置区结构(可简称为环境配置区)可以为用于描述宿主机和容器内的环境设置的数据结构。测试执行数据是软件或硬件在测试过程中产生的所有原始记录和结果信息,用于评估系统功能、性能与稳定性,是质量保障和问题追溯的核心依据。测试执行字段可以是用于描述测试执行数据的相关字段。测试执行描述区结构(可简称为测试执行描述区)可以为用于描述测试命令和结果数据的数据结构。

[0058] 可选的,预设数据结构可以为按芯片测试配置规范所配置的五区结构组织。模板填充引擎可以将实时状态关联信息按照芯片测试配置规范所配置的五区数据结构进行数据填充操作。具体的,模板填充引擎可以从实时状态关联信息中提取元数据字段,并根据元数据字段填充芯片测试配置文件的元数据区结构。示例性的,元数据字段可以包括但不限于version(版本号)、jobid(工作空间ID)、repo(配置项)、job_url(工作空间链接)以及cdash_url等。其中,version字段用于记录配置规范的版本号,用于向后兼容性管理,当规范升级时通过版本号识别;jobid字段为测试任务的唯一标识符,支持变量替换,用于区分不同的测试任务;repo字段为关联的代码仓库地址,用于追溯测试场景对应的源代码版本;job_url字段为关联的芯片测试通用流水线任务的URL(Uniform Resource Locator,统一资源定位符),用于追溯芯片测试配置文件的生成来源;cdash_url字段为测试结果仪表板URL,用于查看测试执行结果和历史记录。

[0059] 具体的,模板填充引擎还可以从实时状态关联信息中提取芯片硬件环境检测字段,并根据芯片硬件环境检测字段填充芯片测试配置文件的硬件环境描述区结构。硬件环境描述区结构的描述数据为区别于通用测试配置文件的核心差异化字段,可以体现芯片测试的特殊性。示例性的,芯片硬件环境检测字段可以包括但不限于target_check_cmd字段和system_check_cmd字段。其中,target_check_cmd字段可以包括芯片检测命令列表,用于验证目标芯片(如目标AI加速卡或通用芯片等)是否存在及其状态,例如可以通过lspci命令检测GCU设备的厂商ID和设备ID。system_check_cmd字段可以包括系统环境检查命令列表,用于验证操作系统挂载点、内核版本等系统级依赖是否满足要求。

[0060] 具体的,模板填充引擎还可以从实时状态关联信息中提取芯片软件环境字段,并根据芯片软件环境字段填充芯片测试配置文件的软件环境描述区结构。示例性的,芯片软件环境字段可以包括但不限于image(容器镜像)、container(容器示例)、package_url(软件包URL)、kmd_install_cmd(驱动安装命令)、docker_launch_cmd(容器启动命令)、container_env_prefix(从外部资源注入到容器的环境变量添加统一的前缀)以及download_cmds(软件包下载命令)等字段。其中,image字段可以包括容器镜像的完整地址

(含仓库路径和版本标签);container字段可以包括容器实例名称,支持变量替换以实现多场景隔离;package_url字段可以包括软件包(如SDK和工具链等)的下载地址;kmd_install_cmd字段可以包括芯片内核驱动模块(Kernel Mode Driver,KMD)的安装命令,安装命令包含驱动安装参数如--driver -y --no-auto-load等;docker_launch_cmd字段可以包括容器启动命令列表,容器启动命令包含挂载路径和设备映射等参数;container_env_prefix字段可以包括容器内命令执行前缀,如docker exec -it {container},用于后续命令的拼接;download_cmds字段可以包括软件包下载命令序列,软件包下载命令序列可按顺序执行以获取所需文件。

[0061] 具体的,模板填充引擎还可以从实时状态关联信息中提取环境配置字段,并根据环境配置字段填充芯片测试配置文件的环境配置区结构。示例性的,环境配置字段可以包括但不限于host_env_setup(宿主机环境设置)字段、container_env_setup(容器内环境设置)字段以及test_env_type(测试环境类型标识)字段。其中,host_env_setup字段可以包括宿主机环境设置命令列表,如设置环境变量和创建目录等;container_env_setup字段可以包括容器内环境设置命令列表,如安装SDK和配置库路径等;test_env_type字段可以包括测试环境类型标识,如dev(development,开发环境)、CI(Continuous Integration,持续集成)和prod(生产环境)等,用于区分不同环境的配置策略。

[0062] 具体的,模板填充引擎还可以从实时状态关联信息中提取测试执行字段,并根据测试执行字段填充芯片测试配置文件的测试执行描述区结构。示例性的,测试执行字段可以包括但不限于test_prefix(测试命令前缀)、tests(测试命令序列)以及corefile(核心转储文件路径)等字段。其中,test_prefix字段可以包括测试命令的公共前缀,测试命令如可以为切换目录以及激活环境等;tests字段可以包括测试命令序列,可按顺序执行各个测试用例;corefile字段可以包括核心转储文件的路径,用于崩溃问题的调试分析。

[0063] 当对预设数据结构的五区结构填充完成后,可以生成dict(一种字典数据结构,本质为键值对集合)结构格式的初始芯片测试配置文件。

[0064] 上述预设数据结构的五区结构设计可以直观反映芯片测试环境的依赖层次即元数据→硬件→软件→环境→执行,每层职责清晰,便于自动化解析。芯片测试配置文件中的target_check_cmd和kmd_install_cmd等字段是针对AI芯片测试场景的专用抽象,填补了通用配置格式的空白。同时,芯片测试配置文件支持{user}和{version}等占位符的自动变量替换机制,可以实现芯片配置模板化和复用。芯片测试配置文件的所有执行类字段均为列表类型,明确了命令的执行顺序和依赖关系。

[0065] S260、对所述初始芯片测试配置文件中的配置数据进行规范化验证。

[0066] 为了进一步保证芯片测试配置文件的可用性和准确性,在生成初始芯片测试配置文件之后,可以进一步对初始芯片测试配置文件中的配置数据进行规范化验证,以验证初始芯片测试配置文件内的配置数据是否符合芯片测试配置的规范。

[0067] 在本发明的一个可选实施例中,所述对所述初始芯片测试配置文件中的配置数据进行规范化验证,可以包括:对所述初始芯片测试配置文件中的必填字段的完整性进行验证;对所述初始芯片测试配置文件中的字段类型的正确性进行验证;对所述初始芯片测试配置文件中的芯片专用字段的有效性进行验证;对所述初始芯片测试配置文件中的测试场景必要字段的覆盖程度进行验证。

[0068] 具体的,配置文件生成模块可以通过模式验证器对初始芯片测试配置文件中的配置数据进行规范化验证,并检查字段完整性和数据类型等。模式验证器可以验证初始芯片测试配置文件中的必填字段的完整性。可选的,必填字段可以包括但不限于`meta.version` (元数据结构版本)、`meta.jobid` (元数据所属工作空间)以及`software.image` (记录包含运行时库的容器镜像的字段)等。同时,模式验证器可以验证初始芯片测试配置文件中的字段类型的正确性。例如,需要验证`tests`字段类型必须是`list`,`kmd_install_cmd`字段类型必须是`string`等。模式验证器还可以验证初始芯片测试配置文件中的芯片专用字段的有效性,如验证`target_check_cmd`字段是否包含AI芯片检测命令等。最后,模式验证器还可以验证初始芯片测试配置文件中的测试场景必要字段的覆盖程度。可选的,测试场景可以包括但不限于核心转储场景、设备测试场景以及库API (Application Programming Interface,应用程序编程接口) 测试场景等。相应的,测试场景必要字段可以是上述各类测试场景所必须配置的字段。

[0069] 相应的,如果初始芯片测试配置文件通过验证,则可以继续后续流程;如果初始芯片测试配置文件未通过验证,则可以根据验证过程中检查出的错误项自动化生成错误报告。

[0070] S270、在确定所述初始芯片测试配置文件通过验证的情况下,将所述初始芯片测试配置文件进行序列化处理,得到所述标准化的芯片测试配置文件。

[0071] 如果确定初始芯片测试配置文件通过验证,配置文件生成模块可以通过文件序列化器进一步对初始芯片测试配置文件进行序列化处理,如将初始芯片测试配置文件的配置数据序列化为JSON和/或YAML格式。在本发明的一个可选实施例中,在文件序列化器的序列化过程中,可以对初始芯片测试配置文件按照设定标准格式进行格式化处理和缩进处理,以提高可读性;还可以对初始芯片测试配置文件添加配置文件生成时间戳和生成系统标识等。文件序列化器对初始芯片测试配置文件经过序列化处理,可以输出标准化的且可读性强的自描述配置文件,从而得到标准化的芯片测试配置文件。

[0072] 可选的,生成的标准化的芯片测试配置文件作为芯片测试通用流水线的输出产物,可以存档到构建产物仓库(Artifact Repository),也还可以通过`job_url`等字段关联到芯片测试通用流水线的测试任务以对目标芯片进行测试,或者也还可以供给配套装置读取并用于芯片测试环境的快速复现等。本发明实施例并不对标准化的芯片测试配置文件的具体应用场景进行限制。

[0073] 在本发明的一个可选实施例中,所述标准化的芯片测试配置文件覆盖的测试场景可以包括:核心转储分析场景、临时文件与工作目录场景、设备测试环境场景以及库API测试场景中的至少一项。也即,标准化的芯片测试配置文件通过其五区结构能够完整覆盖芯片测试的各类关键测试场景。

[0074] 可选的,为了覆盖核心转储分析场景,芯片测试配置文件可以通过`execution.corefile` (一个与查询执行相关的字段,可用于存储中间结果的物理文件) 字段记录核心转储文件的完整路径,如“`/path/to/core.12345`”等;还可以通过`execution.tests` (用于记录测试命令的字段) 字段记录触发崩溃的测试命令,并通过`environment.container_env_setup` (用于初始化或设置容器环境变量的字段) 字段记录调试符号安装命令,如`apt-get install gdb` (用于安装GDB的命令) 等。当将芯片测试配置文

件提供给配套装置复现测试场景时,配套装置可根据上述字段自动下载核心转储文件、重建崩溃环境以及启动gdb(GNU Debugger,是一个由GNU项目开发的强大的程序调试工具)进行堆栈分析。

[0075] 可选的,为了覆盖临时文件与工作目录场景,芯片测试配置文件可以通过`meta.jobid`(元数据中的工作空间字段)字段配置工作目录名称,实现测试场景的相互隔离。芯片测试配置文件还可以通过`software.download_cmds`(用于指定下载软件所需执行的命令列表的字段)字段记录临时文件下载命令,并通过`environment.host_env_setup`(在主机环境中设置环境变量和配置的字段)字段记录临时目录创建命令。当将芯片测试配置文件提供给配套装置复现测试场景时,配套装置可根据`jobid`创建当前测试场景对应的独立工作目录,下载所需临时文件,避免多场景冲突。

[0076] 可选的,为了覆盖设备测试环境场景,芯片测试配置文件可以通过`hardware.target_check_cmd`(用于定义一个命令或脚本的字段,该命令或脚本用于检查硬件目标的状态或性能)字段记录芯片(如AI加速卡)检测命令,如`lspci -d 1e36`:检测GCU设备。芯片测试配置文件还可以通过`hardware.system_check_cmd`(用于定义一个命令或脚本的字段,该命令或脚本用于检查硬件系统的状态或性能)字段记录设备节点检查命令,并通过`software.kmd_install_cmd`(存储内核驱动模式信息的字段)字段记录驱动安装命令及参数。当将芯片测试配置文件提供给配套装置复现测试场景时,配套装置可基于上述字段验证芯片存在性、检查设备节点以及安装正确版本驱动,确保设备可用。

[0077] 可选的,为了覆盖库API测试场景,芯片测试配置文件可以通过`software.image`(软件镜像类字段)字段记录包含运行时库的容器镜像,还可以通过`environment.container_env_setup`(用于控制容器环境的初始化设置的字段)字段记录SDK安装命令,并通过`environment.host_env_setup`字段记录库路径设置,通过`execution.tests`字段记录API测试命令等。当将芯片测试配置文件提供给配套装置复现测试场景时,配套装置可根据上述字段启动正确版本容器、安装SDK、配置库路径以及执行API测试,完整复现库API测试环境。

[0078] 由此可见,芯片测试配置文件在五区结构的设计确保了芯片测试场景的完整性,其中元数据区的配置信息可以提供场景标识和追溯能力,硬件环境描述区的配置信息可以保证硬件依赖可验证,软件环境描述区的配置信息可以保证软件版本可复现,环境配置区的配置信息可以保证环境变量和设置可恢复,测试执行描述区的配置信息可以保证测试用例可重新执行。

[0079] 上述技术方案针对现有测试通用流水线缺乏专门针对芯片测试场景的标准化配置描述语言,导致芯片测试环境难以精确描述和复现的问题,提供了一种专用于描述芯片测试场景的领域特定测试配置文件的生成方法,该方法通过采用分层分区结构对芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息进行填充整理,将芯片测试环境的完整描述信息组织为五个功能区域,每个区域包含若干字段,能够完整且精确地描述芯片软件栈测试所需的硬件环境、软件环境、依赖关系和执行配置,实现芯片测试领域字段约束的结构化规范和芯片测试复现问题领域知识的系统性抽象,能够提高芯片测试配置文件中配置信息的完整性、精确性和标准化程度,有利于实现芯片测试场景的标准化表达和精确复现。

[0080] 具体应用场景

[0081] 为了更清楚的表述本发明实施例提供的技术方案,本发明实施例以CI/CD流水线作为芯片测试通用流水线为例,具体说明本发明实施例提供的芯片测试配置文件生成方法。图3是本发明实施例提供的一种芯片测试配置文件生成系统的架构示意图。在一个具体的例子中,如图3所示,芯片测试配置文件生成系统可以自动化生成芯片测试配置文件。

[0082] 具体的,如图3所示,CI/CD流水线执行过程包含四个执行阶段:环境准备、驱动安装、软件配置以及测试执行,各阶段产生的实时状态数据被实时传递至下方配置实例生成工具的状态捕获模块。CI/CD流水线执行过程包含四个主要阶段,每个阶段完成后,状态捕获模块会自动采集该阶段的状态信息。具体的,在环境准备阶段,系统初始化、检查芯片是否存在以及验证系统依赖。此时状态捕获模块通过硬件信息采集器采集AI芯片硬件信息,并通过环境变量快照器捕获宿主机和容器内的关键环境变量,通过命令执行记录器记录system_check_cmd和target_check_cmd等命令。在驱动安装阶段,安装芯片内核驱动模块,设置驱动参数。此时,状态捕获模块通过软件版本记录器记录驱动版本,通过命令执行记录器记录驱动安装命令及参数等信息。在软件配置阶段,启动容器、安装SDK、运行时库和配置环境变量。此时,状态捕获模块通过软件版本记录器记录容器镜像版本和SDK版本,并通过命令执行记录器记录docker_launch_cmd和container_env_setup等。在测试执行阶段,执行测试用例,并收集测试结果。此时状态捕获模块通过命令执行记录器记录各阶段执行的测试命令序列(target_check_cmd和kmd_install_cmd等)以及核心转储文件路径信息等。状态捕获模块的数据采集功能整合在CI/CD流水线执行阶段中。

[0083] 配置实例生成工具的配置文件生成器通过模板填充引擎接收状态捕获模块发送的实时状态关联信息,对其提取相应的关键信息填充至芯片测试配置数据规范的五区结构(元数据区、硬件环境描述区、软件环境描述区、环境配置区、测试执行描述区)组织数据对应字段中,得到初始芯片测试配置文件。例如,模板填充引擎将驱动安装命令填充到software.kmd_install_cmd字段,将芯片检测命令填充到hardware.target_check_cmd字段等。进一步的,配置文件生成器通过模式验证器验证初始芯片测试配置文件中的配置数据是否符合芯片测试领域特定规范。最后,配置文件生成器通过文件序列化器将验证后的初始芯片测试配置文件序列化为JSON或YAML格式的标准化的芯片测试配置文件并输出。

[0084] 相应的,芯片测试配置文件生成系统自动生成输出的芯片测试配置文件属于一种自描述配置文件,遵循芯片测试领域特定规范,字段定义明确,结构统一,包含完整的环境信息,无需额外的文档或说明。芯片测试配置文件不限定平台类型,可应用在不同平台复现相应的芯片测试场景。芯片测试配置文件版本可追溯,可通过repo或job_url等元数据字段,追溯到源代码版本和对应的流水线任务。在生成芯片测试配置文件的过程中,芯片测试配置文件生成系统可以仅关注流水线执行测试任务所需的处理器架构、网络配置、软件版本以及硬件状态,而无需关注机器从哪个集群分配、日志如何上传以及任务如何调度等无关信息。

[0085] 与现有技术相比,上述芯片测试配置文件生成方法能够在CI/CD流水线执行过程中自动采集芯片测试场景的全量状态信息,并按照芯片领域特定的配置规范生成标准化的、自描述性的且格式统一的芯片测试配置文件。可选的,芯片测试配置文件生成系统自动生成输出的芯片测试配置文件可以作为“设计者-使用者”之间的契约接口,可被外部解析

装置读取,并用于芯片测试环境的快速准确复现。可选的,芯片测试配置文件生成系统可在任意CI/CD流水线平台上部署,生成的自描述性的芯片测试配置文件格式统一,与具体流水线实现解耦。

[0086] 在上述芯片测试配置文件生成系统中,CI/CD测试流水线可由流水线设计者(也可以是配置文件的生产者)负责流水线的设计和维护。流水线设计者可以定义测试场景,如选择测试用例、配置测试参数以及设置运行环境等。流水线执行时,芯片测试配置文件生成系统自动采集CI/CD测试流水线各阶段的全量状态信息,生成输出芯片测试配置文件。芯片测试配置文件可以包括复现该测试场景所需的完整信息。相应的,开发工程师、测试工程师或技术支持工程师等任意工程师作为问题排查者(也可以是配置文件的消费者),需要在本地环境复现CI/CD流水线中出现的问题(如测试失败和程序崩溃等)时,可以获取芯片测试配置文件生成系统生成的标准化的芯片测试配置文件,将芯片测试配置文件提供给配套装置解析执行,实现一键复现芯片专用的测试环境。

[0087] 图4是本发明实施例提供的一种芯片测试配置文件的数据结构示意图。在一个具体的例子中,如图4所示,芯片测试配置文件配置规范的数据结构布局,自上而下依次为:元数据区,包括版本、任务标识、仓库与流水线链接等字段;硬件环境描述区,包括芯片检测与系统检查命令等字段;软件环境描述区,包括镜像、容器、驱动安装与容器启动等字段;环境配置区,包括宿主机与容器内环境设置等字段;测试执行描述区,包括测试命令前缀、测试列表以及核心转储路径等字段。各区字段与芯片测试领域的特定约束一一对应,供解析系统自动化识别并执行。

[0088] 可选的,如图4所示,芯片测试配置文件还可以根据特定类型芯片的测试需求适应扩展其数据结构,如增设依赖描述区结构(简称为依赖描述区,dependencies)等。示例性的,依赖描述区结构可以配置的字段包括但不限于framework、framework_version、model_artifacts以及dataset_paths等。其中,framework字段可以用于指定开发框架或系统框架的类型,framework_version可以用于指定开发框架或系统框架的版本号。model_artifacts为适用于AI芯片类型的字段,用于配置模型训练、编译或部署过程中生成的所有关键文件和资源。dataset_paths字段指的是指向数据集文件或文件夹的路径。

[0089] 在本发明的一个可选实施例中,如下为芯片测试配置文件(ChipTestConfig)的形式化定义示例:

```
[0090] ChipTestConfig {
[0091]   // 元数据区-支持版本控制和溯源
[0092]   meta {
[0093]     version: string //配置规范版本号
[0094]     jobid: string //任务唯一标识(支持变量替换如{user}_{version})
[0095]     repo: string //关联的代码仓库
[0096]     job_url: string //关联的CI/CD流水线URL
[0097]     cdash_url: string //测试结果仪表板URL
[0098]   }
[0099]   // 硬件环境描述区-芯片领域特有专用字段(核心差异化)
[0100]   hardware {
```

```
[0101]     target_check_cmd: list //芯片检测命令(如lspci检测GCU设备)
[0102]     system_check_cmd: list //系统环境检查命令
[0103] }
[0104] // 软件环境描述区-芯片驱动和运行时特有配置
[0105] software {
[0106]     image: string //容器镜像地址
[0107]     container: string //容器名称(支持变量替换)
[0108]     package_url: string //软件包下载地址
[0109]     download_cmds: list //软件包下载命令序列
[0110]     kmd_install_cmd: string //芯片特有:驱动安装命令(含参数如--driver
-y --no-auto-load)
[0111]     docker_launch_cmd: list //容器启动命令
[0112]     container_env_prefix: string //容器命令执行前缀
[0113] }
[0114] // 环境配置区-宿主机和容器环境设置
[0115] environment {
[0116]     host_env_setup: list //宿主机环境设置命令
[0117]     container_env_setup: list //容器内环境设置命令(如SDK安装)
[0118]     test_env_type: string //测试环境类型标识
[0119] }
[0120] // 测试执行描述区-支持芯片测试场景的完整描述
[0121] execution {
[0122]     test_prefix: string //测试命令前缀
[0123]     tests: list //测试命令序列
[0124]     corefile: string //核心转储文件路径(用于崩溃分析)
[0125] }
[0126] }
[0127] 在一个具体的例子中,如下示例展示了芯片测试配置文件的五区结构如何完整地
描述一个AI芯片测试场景的所有环境依赖,使得任何工程师都可以基于该配置文件在本地
精确复现CI/CD流水线中的AI芯片测试环境。
[0128] {
[0129]     "version": "1.0",
[0130]     "repo": "ai-test",
[0131]     "jobid": "{user}_topsinference_{version}",
[0132]     "image": "ab.cn/enflame_docker_images/ubuntu/test_2004:1.1.1",
[0133]     "container": "{user}_topsinference_{version}",
[0134]     "job_url": "http://aaaaab/job/12345",
[0135]     "cdash_url": "http://aaaaac.com/test/12345",
[0136]     "package_url": "http://aaaaad.com/release/3.6.105/TopsRider.run",
```

```
[0137]   "download_cmds": [  
[0138]     "wget -c --tries=5 {package_url}"  
[0139]   ],  
[0140]   "system_check_cmd": [  
[0141]     "mount | grep -E '.devtools|.distdir|tops'"  
[0142]   ],  
[0143]   "target_check_cmd": [  
[0144]     "lspci -d 1e36: | grep -i -E 'c033|c035|s60'"  
[0145]   ],  
[0146]   "host_env_setup": [],  
[0147]   "kmd_install_cmd": "ar*.run --driver -y --no-auto-load",  
[0148]   "test_env_type": "inference",  
[0149]   "docker_launch_cmd": [  
[0150]     "efdocker run ci2004 --name {container}"  
[0151]   ],  
[0152]   "container_env_prefix": "docker exec -t {container} bash -c",  
[0153]   "container_env_setup": [  
[0154]     "sudo bash TopsRider*.run --container -C TopsInference -y"  
[0155]   ],  
[0156]   "corefile": "",  
[0157]   "test_prefix": "",  
[0158]   "tests": [  
[0159]     "topsinfo",  
[0160]     "python run_inference.py --model resnet50 --batch-size 32"  
[0161]   ]  
[0162] }
```

[0163] 表1 芯片测试配置文件的核心技术特征

[0164]	特征	说明	与现有技术的区别
	AI 芯片硬件检测字段	target_check_cmd 字段用于检测 AI 加速卡（如通过 lspci 检测 GCU 设备）	传统测试配置不包含 AI 芯片检测逻辑
	驱动层结构抽象	kmd_install_cmd 字段专门描述 AI 芯片驱动安装命令和参数	通用测试框架无此抽象层
	容器化环境配置	docker_launch_cmd、container_env_setup 字段支持容器化测试环境	针对 AI 芯片测试的容器化需求
	分区结构设计	元数据-硬件-软件-环境-执行五层分离，反映 AI 芯片测试的实际依赖关系	现有配置文件缺乏此领域结构
	变量替换机制	支持 {user}、{version} 等变量替换，实现配置模板化	实现配置的复用性和灵活性

[0165] 表1是本发明实施例提供的一种芯片测试配置文件的核心技术特征列表,如表1所示,本发明实施例提供的芯片测试配置文件实现了对芯片专用测试环境描述问题的领域知识系统性抽象,填补了芯片测试场景缺乏标准化配置描述语言的空白。

[0166] 综上所述,通过自动化生成的芯片测试配置文件,非流水线设计者无需了解流水线内部实现,即可快速精准复现虚芯片测试场景,从而降低复现门槛。问题排查者通过芯片测试配置文件即可获取完整的环境信息,从而减少沟通成本。同时,芯片测试配置文件标准化的配置格式能够确保环境配置的完整性和一致性,从而提高测试复现的成功率。同一芯片测试配置文件可在不同流水线平台间复用,支持跨平台迁移。芯片测试配置文件特定的数据模式格式和字段结构具有可识别性,程序或测试工具可直接按固定格式自动读取配置,无需人工逐行修改适配,实现自动化加载,且可降低配置写错、漏填及错填概率,复用性强,兼容扩展升级。

[0167] 需要说明的是,本公开所涉及的相关信息(包括但不限于用户设备信息、用户个人信息等)和数据(包括但不限于用于展示的数据、分析的数据等),均为经用户授权或者经过各方充分授权的信息和数据,且相关数据的收集、使用和处理遵守相关地区的相关法律法规和标准。

[0168] 需要说明的是,以上各实施例中各技术特征之间的任意排列组合也属于本发明的保护范围。

[0169] 实施例三

[0170] 图5是本发明实施例三提供的一种芯片测试配置文件生成装置的示意图,如图5所示,所述装置包括:实时状态关联信息采集模块510以及芯片测试配置文件生成模块520,其中:

[0171] 实时状态关联信息采集模块510,用于采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;

[0172] 芯片测试配置文件生成模块520,用于根据所述实时状态关联信息生成标准化的芯片测试配置文件。

[0173] 本发明实施例通过采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息,从而根据采集的实时状态关联信息生成标准化的芯片测试配置文件,解决现有芯片测试场景中通用配置文件无法满足芯片专用测试需求的问题,能够提高芯片测试配置文件中配置信息的完整性、精确性和标准化程度,有利于实现芯片测试场景的标准化表达和精确复现。

[0174] 可选的,实时状态关联信息采集模块510还用于:在所述芯片测试通用流水线的环境准备阶段,采集芯片设备硬件信息;在所述芯片测试通用流水线的驱动安装阶段,采集芯片软件版本关联信息;在所述芯片测试通用流水线的软件配置阶段,采集环境变量快照信息;在所述芯片测试通用流水线的测试执行阶段,采集测试命令执行记录信息。

[0175] 可选的,芯片测试配置文件生成模块520还用于:根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,得到初始芯片测试配置文件;对所述初始芯片测试配置文件中的配置数据进行规范化验证;在确定所述初始芯片测试配置文件通过验证的情况下,将所述初始芯片测试配置文件进行序列化处理,得到所述标准化的芯片测试配置文件。

[0176] 可选的,芯片测试配置文件生成模块520还用于:从所述实时状态关联信息中提取元数据字段,并根据所述元数据字段填充所述芯片测试配置文件的元数据区结构;从所述实时状态关联信息中提取芯片硬件环境检测字段,并根据所述芯片硬件环境检测字段填充所述芯片测试配置文件的硬件环境描述区结构;从所述实时状态关联信息中提取芯片软件环境字段,并根据所述芯片软件环境字段填充所述芯片测试配置文件的软件环境描述区结构;从所述实时状态关联信息中提取环境配置字段,并根据所述环境配置字段填充所述芯片测试配置文件的环境配置区结构;从所述实时状态关联信息中提取测试执行字段,并根据所述测试执行字段填充所述芯片测试配置文件的测试执行描述区结构。

[0177] 可选的,芯片测试配置文件生成模块520还用于:对所述初始芯片测试配置文件中的必填字段的完整性进行验证;对所述初始芯片测试配置文件中的字段类型的正确性进行验证;对所述初始芯片测试配置文件中的芯片专用字段的有效性进行验证;对所述初始芯片测试配置文件中的测试场景必要字段的覆盖程度进行验证。

[0178] 可选的,芯片测试配置文件生成模块520还用于:将所述初始芯片测试配置文件按照设定标准格式进行格式化处理和缩进处理;对所述初始芯片测试配置文件添加配置文件生成时间戳和生成系统标识。

[0179] 可选的,所述标准化的芯片测试配置文件覆盖的测试场景包括:核心转储分析场景、临时文件与工作目录场景、设备测试环境场景以及库应用程序接口API测试场景中的至少一项。

[0180] 上述芯片测试配置文件生成装置可执行本发明任意实施例所提供的芯片测试配置文件生成方法,具备执行方法相应的功能模块和有益效果。未在本实施例中详尽描述的技术细节,可参见本发明任意实施例提供的芯片测试配置文件生成方法。

[0181] 由于上述所介绍的芯片测试配置文件生成装置为可以执行本发明实施例中的芯片测试配置文件生成方法的装置,故而基于本发明实施例中所介绍的芯片测试配置文件生成方法,本领域所属技术人员能够了解本实施例的芯片测试配置文件生成装置的具体实施方式以及其各种变化形式,所以在此对于该芯片测试配置文件生成装置如何实现本发明实

施例中的芯片测试配置文件生成方法不再详细介绍。只要本领域所属技术人员实施本发明实施例例中芯片测试配置文件生成方法所采用的装置,都属于本申请所欲保护的范围。

[0182] 实施例四

[0183] 图6示出了可以用来实施本发明的实施例的电子设备10的结构示意图。电子设备旨在表示各种形式的数字计算机,诸如,膝上型计算机、台式计算机、工作台、个人数字助理、服务器、刀片式服务器、大型计算机、和其它适合的计算机。电子设备还可以表示各种形式的移动装置,诸如,个人数字处理、蜂窝电话、智能电话、可穿戴设备(如头盔、眼镜、手表等)和其它类似的计算装置。本文所示的部件、它们的连接和关系、以及它们的功能仅仅作为例,并且不意在限制本文中描述的和/或者要求的本发明的实现。

[0184] 如图6所示,电子设备10包括至少一个处理器11,以及与至少一个处理器11通信连接的存储器,如只读存储器ROM 12、随机访问存储器RAM 13等,其中,存储器存储有可被至少一个处理器执行的计算机程序,处理器11可以根据存储在只读存储器ROM 12中的计算机程序或者从存储单元18加载到随机访问存储器RAM 13中的计算机程序,来执行各种适当的动作和处理。在RAM 13中,还可存储电子设备10操作所需的各种程序和数据。处理器11、ROM 12以及RAM 13通过总线14彼此相连。输入/输出(I/O)接口15也连接至总线14。

[0185] 电子设备10中的多个部件连接至I/O接口15,包括:输入单元16,例如键盘、鼠标等;输出单元17,例如各种类型的显示器、扬声器等;存储单元18,例如磁盘、光盘等;以及通信单元19,例如网卡、调制解调器、无线通信收发机等。通信单元19允许电子设备10通过诸如因特网的计算机网络和/或各种电信网络与其他设备交换信息/数据。

[0186] 处理器11可以是各种具有处理和计算能力的通用和/或专用处理组件。处理器11的一些示例包括但不限于中央处理单元(CPU)、图形处理单元(GPU)、各种专用的人工智能(AI)计算芯片、各种运行机器学习模型算法的处理器、数字信号处理器(DSP)、以及任何适当的处理器、控制器、微控制器等。处理器11执行上文所描述的各个方法和处理,例如芯片测试配置文件生成方法。

[0187] 可选的,芯片测试配置文件生成方法可以包括:采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;根据所述实时状态关联信息生成标准化的芯片测试配置文件。

[0188] 在一些实施例中,芯片测试配置文件生成方法可被实现为计算机程序,其被有形地包含于计算机可读存储介质,例如存储单元18。在一些实施例中,计算机程序的部分或者全部可以经由ROM 12和/或通信单元19而被载入和/或安装到电子设备10上。当计算机程序加载到RAM 13并由处理器11执行时,可以执行上文描述的芯片测试配置文件生成方法的一个或多个步骤。备选地,在其他实施例中,处理器11可以通过其他任何适当的方式(例如,借助于固件)而被配置为执行芯片测试配置文件生成方法。

[0189] 本文中以上描述的系统和技术各种实施方式可以在数字电子电路系统、集成电路系统、场可编程门阵列(FPGA)、专用集成电路(ASIC)、专用标准产品(ASSP)、芯片上系统的系统(SOC)、负载可编程逻辑设备(CPLD)、计算机硬件、固件、软件、和/或它们的组合中实现。这些各种实施方式可以包括:实施在一个或者多个计算机程序中,该一个或者多个计算机程序可在包括至少一个可编程处理器的可编程系统上执行和/或解释,该可编程处理器可以是专用或者通用可编程处理器,可以从存储系统、至少一个输入装置、和至少一个输出

装置接收数据和指令,并且将数据和指令传输至该存储系统、该至少一个输入装置、和该至少一个输出装置。

[0190] 用于实施本发明的方法的计算机程序可以采用一个或多个编程语言的任何组合来编写。这些计算机程序可以提供给通用计算机、专用计算机或其他可编程数据处理装置的处理器,使得计算机程序当由处理器执行时使流程图和/或框图所示的功能/操作被实施。计算机程序可以完全在机器上执行、部分地在机器上执行,作为独立软件包部分地在机器上执行且部分地在远程机器上执行或完全在远程机器或服务器上执行。

[0191] 在本发明的上下文中,计算机可读存储介质可以是有形的介质,其可以包含或存储以供指令执行系统、装置或设备使用或与指令执行系统、装置或设备结合地使用的计算机程序。计算机可读存储介质可以包括但不限于电子的、磁性的、光学的、电磁的、红外的、或半导体系统、装置或设备,或者上述内容的任何合适组合。备选地,计算机可读存储介质可以是机器可读信号介质。机器可读存储介质的更具体示例会包括基于一个或多个线的电气连接、便携式计算机盘、硬盘、随机存取存储器(RAM)、只读存储器(ROM)、可擦除可编程只读存储器(EPROM或快闪存储器)、光纤、便捷式紧凑盘只读存储器(CD-ROM)、光学储存设备、磁储存设备、或上述内容的任何合适组合。

[0192] 为了提供与用户的交互,可以在电子设备上实施此处描述的系统和技术,该电子设备具有:用于向用户显示信息的显示装置(例如,CRT(阴极射线管)或者LCD(液晶显示器)监视器);以及键盘和指向装置(例如,鼠标或者轨迹球),用户可以通过该键盘和该指向装置来将输入提供给电子设备。其它种类的装置还可以用于提供与用户的交互;例如,提供给用户的反馈可以是任何形式的传感反馈(例如,视觉反馈、听觉反馈、或者触觉反馈);并且可以用任何形式(包括声输入、语音输入或者触觉输入)来接收来自用户的输入。

[0193] 可以将此处描述的系统和技术实施在包括后台部件的计算系统(例如,作为数据服务器)、或者包括中间件部件的计算系统(例如,应用服务器)、或者包括前端部件的计算系统(例如,具有图形用户界面或者网络浏览器的用户计算机,用户可以通过该图形用户界面或者该网络浏览器来与此处描述的系统和技术实施方式交互)、或者包括这种后台部件、中间件部件、或者前端部件的任何组合的计算系统中。可以通过任何形式或者介质的数字数据通信(例如,通信网络)来将系统的部件相互连接。通信网络的示例包括:局域网(LAN)、广域网(WAN)、区块链网络和互联网。

[0194] 计算系统可以包括客户端和服务端。客户端和服务端一般远离彼此并且通常通过通信网络进行交互。通过在相应的计算机上运行并且彼此具有客户端-服务器关系的计算机程序来产生客户端和服务端的关系。服务器可以是云服务器,又称为云计算服务器或云主机,是云计算服务体系中的一项主机产品,以解决了传统物理主机与VPS服务中,存在的管理难度大,业务扩展性弱的缺陷。

[0195] 本发明实施例还公开了一种计算机程序产品,所述计算机程序产品包括计算机程序,所述计算机程序在被处理器执行时实现本发明任一实施例所提供的芯片测试配置文件生成方法。该程序产品与本发明各实施例所公开的芯片测试配置文件生成方法属于相同发明构思,因此不在此赘述。

[0196] 应该理解,可以使用上面所示的各种形式的流程,重新排序、增加或删除步骤。例如,本发明实施例中记载的各步骤可以并行地执行也可以顺序地执行也可以不同的次序执

行,只要能够实现本发明实施例公开的技术方案所期望的结果,本文在此不进行限制。

[0197] 上述具体实施方式,并不构成对本公开保护范围的限制。本领域技术人员应该明白的是,根据设计要求和其他因素,可以进行各种修改、组合、子组合和替代。任何在本公开的精神和原则之内所作的修改、等同替换和改进等,均应包含在本公开保护范围之内。

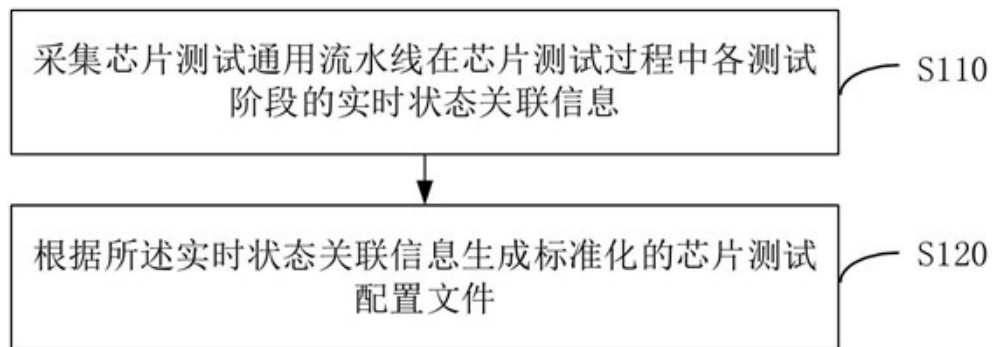


图1

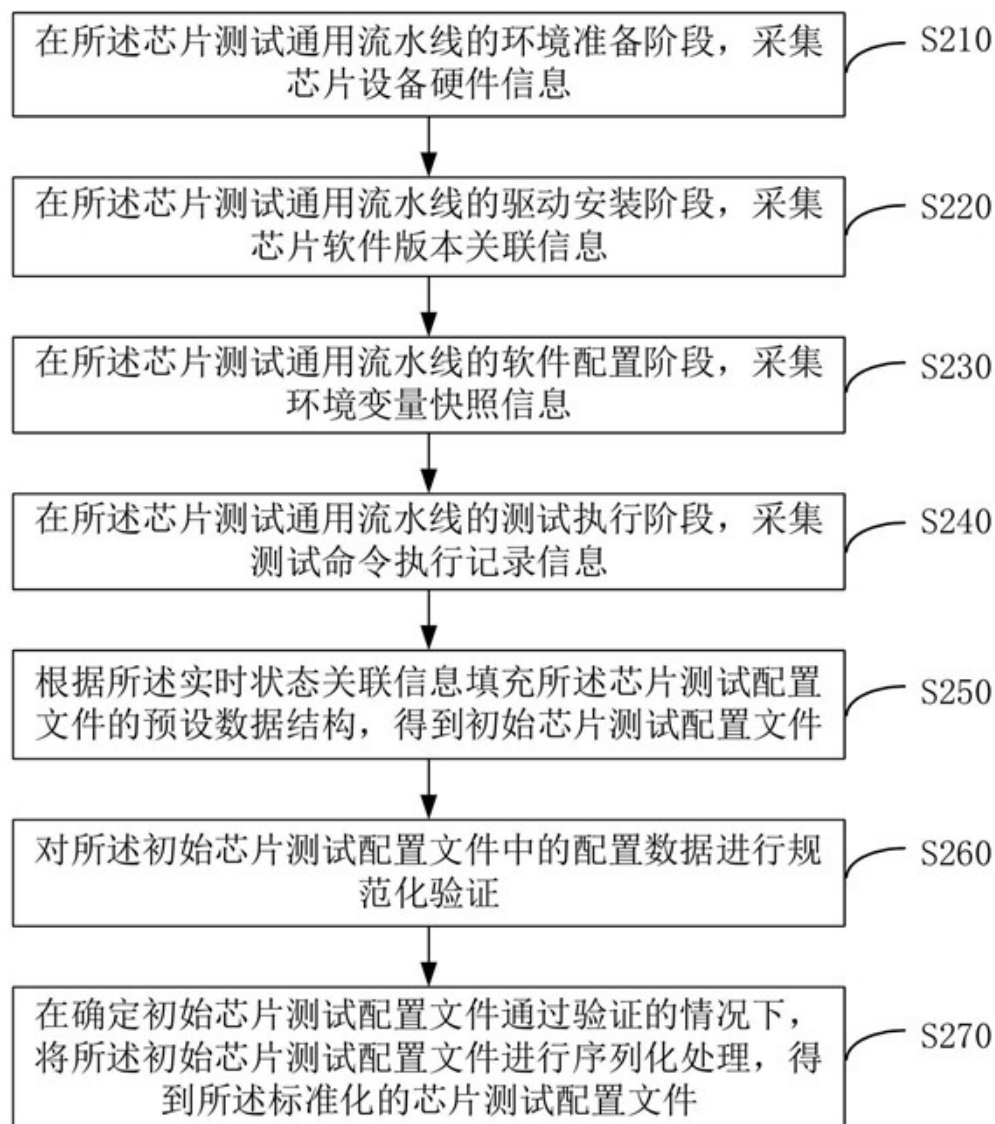


图2

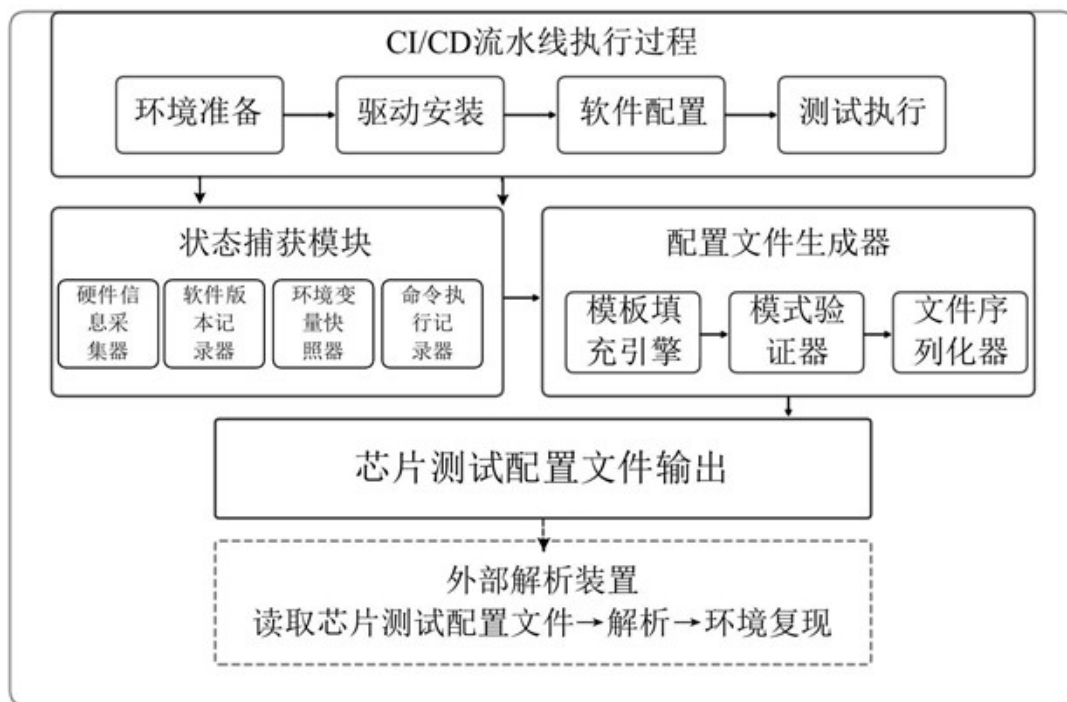


图3



图4

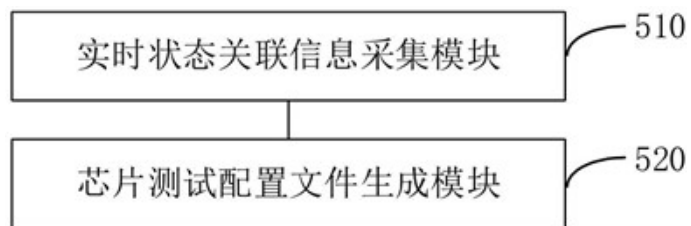


图5

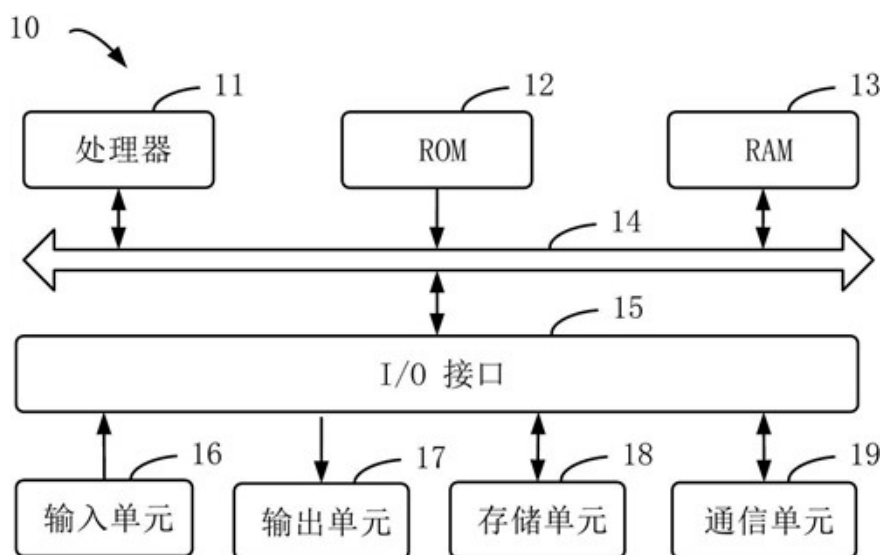


图6